AeroParse: Knowledge Distillation & Deployment on Local Cloud

CS3081 Cloud Computing — Assignment Mini Project Report | Roll No: 230101065

System Abstract & Overview

This project investigates sequence-level knowledge distillation and cloud-native deployment of an ultra-lightweight command parser. A 3.8-billion parameter teacher model (Microsoft Phi-3.5-mini-instruct via Ollama) was distilled into a 70-million parameter student model (EleutherAI Pythia-70M). The model is served on a local Kubernetes cluster (k3d) with SeaweedFS S3 object storage, horizontal pod autoscaling, health probes, and automated CI/CD testing. All seven tasks specified in the assignment are addressed with measured evidence, architectural diagrams, and critical reflection.

Task 1: Object Storage and Parallel Teacher Labelling

Cluster & Storage Architecture: A single-node local Kubernetes cluster was provisioned using k3d from cluster/k3d.yaml with port 8080 forwarded to the Traefik ingress load balancer. SeaweedFS was deployed in the data namespace via Helm (cluster/seaweedfs-values.yaml) exposing an S3-compatible API with buckets datasets and models. The input pool commands_pool.jsonl (3,000 commands) was uploaded to the datasets bucket.

Parallel Sharding Architecture: The teacher (Phi-3.5) was served locally via Ollama with fixed seed 42 and temperature 0.0. To accelerate labelling, worker/teacher_worker.py sharded commands across worker pods using modulo arithmetic: (line_index % SHARD_COUNT) == SHARD_INDEX. Two workers ran in namespace pipeline, writing datasets/teacher/shard-0.jsonl and shard-1.jsonl. Every command ID appeared in exactly one shard.

Data Cleaning Pipeline: worker/clean.py loaded all shard outputs and rejected invalid entries according to the five assignment criteria: (a) malformed JSON, (b) invalid key sets (strictly: intent, date, time, place), (c) unallowed intents, (d) unanchored slot values not present in command text, and (e) duplicate commands (retaining lowest ID).

Category	Filter Rule	Count
Input Records	Total pool from commands_pool.jsonl	3,000
Rule (a) Invalid JSON	Syntax error or decode failure	142
Rule (b) Wrong Keys	Keys != {intent, date, time, place}	88
Rule (c) Invalid Intent	Intent outside 8 allowed categories	64
Rule (d) Invalid Slot	Slot text not verbatim in command	118
Rule (e) Duplicates	Duplicate commands removed	296
Kept Records	Cleaned training dataset (datasets/v1/train.jsonl)	**2,292**

Integrity Formula Verification: 3,000 = 142 + 88 + 64 + 118 + 296 + 2,292 (exact match). SHA-256 recorded in train.sha256.

Task 2: Training the Student (Sequence-Level Distillation)

Training Setup: EleutherAI/pythia-70m was fine-tuned on CPU using PyTorch and Hugging Face Transformers on the first 1,500 records of train.jsonl. Each sequence was structured as Command: <text>\nJSON: <teacher_reply>. Loss was computed solely on JSON generation tokens by masking prompt tokens with -100.

Reproducibility: Configured via training/config.yaml with pinned seed (42), learning rate (5e-5), batch size (4), epochs (3), max sequence length (256), and library versions (Python 3.11, PyTorch 2.15.0, Transformers 4.57.6). Supports single-command execution via make train and verification subset make train SUBSET=50. Loss per epoch logged to training/loss.csv.

Model Artefacts: Model v1 and v2 uploaded to S3 bucket models under models/v1/ and models/v2/. Checksum saved in models/v1/model.sha256 matches the published GitHub Release asset.

Metric	Teacher (Phi-3.5)	Untrained Pythia-70M	Distilled Student (v1)
JSON Validity	94.0%	0.0%	90.0%
Intent Accuracy	85.0%	0.0%	75.0%
Date Accuracy	71.0%	0.0%	68.0%
Time Accuracy	79.0%	0.0%	76.0%
Place Accuracy	81.0%	0.0%	77.0%
Full-Frame Accuracy	**44.0%**	**0.0%**	**41.0%**

Theoretical Explanation: (a) *Sequence-level distillation* trains on teacher generated discrete token sequences using standard cross-entropy loss. (b) *Logit-based distillation* trains on soft probability distributions across the full vocabulary via KL divergence, transferring dark knowledge. (c) *Why logit-based is impossible here:* Ollama exposes a black-box text-generation API without logit access, and Phi-3.5 (32K vocabulary) and Pythia-70M (50K vocabulary) share incompatible tokenizers. (d) *Limitation:* Sequence-level distillation provides no uncertainty signal; the student memorizes teacher hallucinations and syntax errors equally with correct samples.

Task 3: The Inference Service in a Container

FastAPI Microservice: Implemented in service/app.py with endpoints: POST /parse returning four-key JSON and headers X-Model-Version and X-Served-By; GET /healthz (200 alive); GET /readyz (503 loading, 200 ready). Configuration strictly governed by MODEL_URI, MODEL_VERSION, S3_ENDPOINT, and MAX_NEW_TOKENS.

Dynamic S3 Loading & Secrets: Container image contains zero weights. At startup, weights are downloaded from SeaweedFS using S3 credentials mounted from Kubernetes Secret seaweedfs-s3-credentials.

Schema Validation & Fault Injection: Every output is validated against schema.json. Malformed JSON or schema violations return HTTP 500 with {"error": "invalid_model_output"}. Setting FAULT_INJECT=1 forces output to 'not json', verifying error interception.

Container Optimization & Resource Limit Test: Built on python:3.12-slim with CPU-only wheels, image size is 1.66 GB (< 2 GB). 50 sequential requests under 1 CPU / 1 GiB memory limits completed with 0 OOM kills and **737.4 MiB peak memory**.

Task 4: Kubernetes Deployment, Scaling, and Reliability

Workload Orchestration: Deployed in namespace app with 2 replicas, resource requests 500m/512Mi, limits 1 CPU/1GiB, ClusterIP service, Traefik ingress for /parse, and readiness/liveness probes.

Autoscaling (HPA): HPA configured for min 1, max 2 replicas at 70% CPU target. Scaled to 2 replicas under load, then scaled down to 1 after load stopped.

Rolling Updates & Rollback: Updating MODEL_VERSION from v1 to v2 during active load achieved 0.0% failed requests. Rollback via kubectl rollout undo restored v1 without downtime.

Recovery & Cold-Start: Pod deletion under load recovered replica count with < 2% request drops. Cold-start median across 3 trials was **42.5 seconds** (Run 1: 43.8s, Run 2: 41.2s, Run 3: 42.5s).

Load Balancing: Over 2-minute load test, Pod 1 served 41 requests (52.6%) and Pod 2 served 37 requests (47.4%), satisfying the >30% requirement.

Task 5: Subnetting and Network Access Control

VLSM Subnet Allocation: Address block 10.20.0.0/16 divided contiguously from largest requirement to smallest using usable_hosts = 2^(32 - prefix) - 2:

Subnet	Required	Network Address	Prefix	Usable Host Range	Broadcast	Usable Hosts
app	1,000	10.20.0.0	/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1,022
public	250	10.20.4.0	/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0	/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128	/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

NACL Implementation: Public subnet stateless NACL (network/nacl.json) tested with tools/nacl_sim.py (6/6 = 100.0% correct). **Operational Reasoning:** (1) Stateless NACLs do not track connection state; return traffic to ephemeral ports (1024-65535) requires separate explicit rules. (2) Rules evaluate in ascending order with first match winning; deny rule for 198.51.100.0/24 must precede broad allow rules to prevent bypass.

Task 6: Reproducibility, Testing, and CI/CD

Automation: Single-command deployment via make up and teardown via make down. **GitHub Actions Pipeline:** Four strictly ordered stages: (1) Linting (Ruff and compileall), (2) Unit tests (17 tests across 4 modules), (3) Model evaluation downloading release asset with quality gate score.py --min-validity 95, and (4) Secret scanning via Gitleaks (0 leaks). Deliberately evaluated untrained Pythia-70M failed the gate (0% validity), proving pipeline protection. AI_USE.md documents tool attribution.

Task 7: Performance, Cost, and Review

Locust Load Test Measurements: 4 users, 2 minutes: 89 requests, 78 successful, 11 failed (error rate 12.36%). Throughput: 0.748 req/s. Median latency (p50): 3,700 ms. 95th percentile latency (p95): 16,000 ms. CPU peaked near 1 core limit; RAM peaked at 560 MiB.

Cost Estimation (rate_card.json): Replica rate = 1 vCPU (3.50 INR) + 1 GiB RAM (0.45 INR) = 3.95 INR/hr. Two replicas = 7.90 INR/hr. Hourly requests served = (78 / 120s) * 3600 = 2,340 req/hr. **Cost per 1,000 requests = (7.90 / 2340) * 1000 = 3.376 INR**.

Service Level Objective (SLO): Target: 99.0% success over 2-minute load test. Total requests: 89. Error budget: floor(89 * 0.01) = 0 allowed failures. Actual failures: 11. Error budget consumed: **1,236%** (SLO breached due to CPU saturation on laptop).

Component	Measured Evidence	Identified Limitation	Proposed Improvement
Data & Storage	2,292 valid records from 3,000 outputs	Single SeaweedFS volume; no HA replication	Multi-AZ MinIO cluster with 3x replication factor
Model Quality	90.0% JSON validity; 41.0% full-frame	Occasional truncated JSON syntax errors	Constrained grammar decoding (Outlines/Guidance)
Latency & Throughput	0.748 req/s; p95 latency = 16,000 ms	CPU-only unquantized autoregressive decoding	INT8 ONNX Runtime or GGML/llama.cpp quantization
Resource Utilisation	Peak RAM 560 MiB; CPU throttled at 1 core	CPU saturation under concurrency causes queuing	OpenMP core affinity and dedicated cloud compute
Reliability & Recovery	Cold-start median 42.5s; pod recovered	Synchronous S3 download delays pod readiness	Local hostPath node caching or CSI volume snapshots
Security & Cost	Cost: 3.376 INR/1k req; 6/6 NACL passed	Ingress lacks TLS termination (plain HTTP)	cert-manager Let's Encrypt TLS and Istio mTLS